

Session #13-14:

Object Detection (with introduction to RCNN, YOLO)

Dhruba Adhikary

Dhruba.a@wilp.bits-pilani.ac.in

Topics

- Object Detection & Challenges
- RCNN & Faster RCNN
- Yolo & Its Variants
- Demonstration

Acknowledgement: Slide Materials adopted from - Intro to Computer Vision (Cornell Tech); Noah Snavely

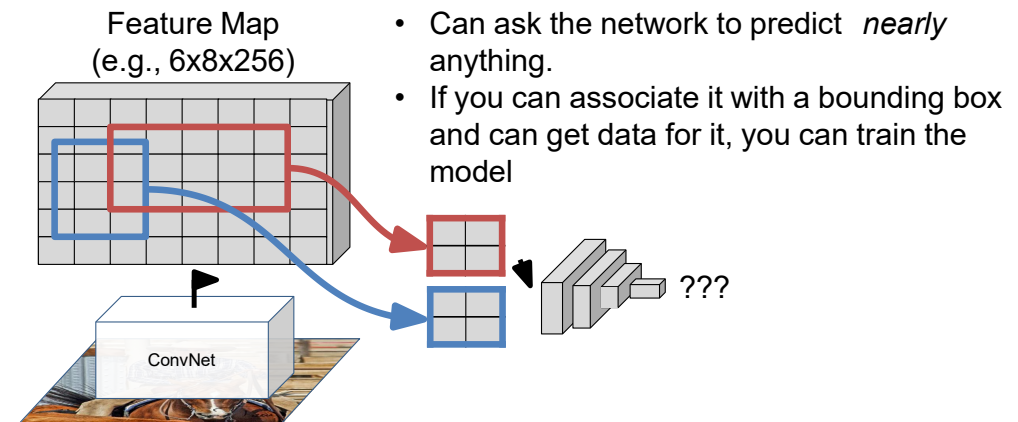
1

2

Key Differences:

Feature	RCNN	Fast RCNN	Faster RCNN	Mask RCNN
Region Proposal	External methods (Selective Search)	External methods (Selective Search)	Integrated Region Proposal Network (RPN)	Integrated RPN with mask prediction
Speed	Slow	Faster than RCNN	Faster than Fast RCNN	Slightly slower than Faster RCNN due to mask branch
Architecture	Multi-stage	End-to-end with external region proposals	Fully end-to-end with shared features	Fully end-to-end with shared features and mask prediction
Output	Bounding boxes	Bounding boxes	Bounding boxes	Bounding boxes + segmentation masks
Training Complexity	High	Lower than RCNN	Single unified model	Similar to Faster RCNN but with additional mask training
Use Case	Basic object detection	Faster object detection	Real-time object detection	Object detection + instance segmentation

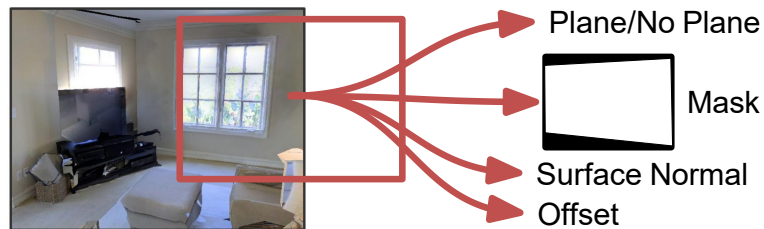
Extending Object Detection



- Can ask the network to predict *nearly* anything.
- If you can associate it with a bounding box and can get data for it, you can train the model

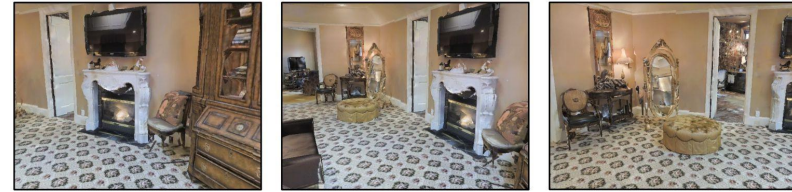
Extending Object Detection

Example: RGB image input, detect planar surfaces



Liu et al. *PlaneRCNN: 3D Plane Detection and Reconstruction from a Single Image*. CVPR 2019.

Extending Object Detection

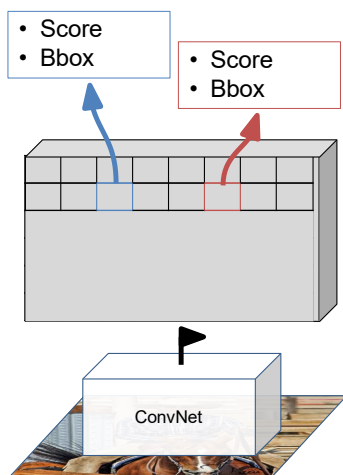


Core building block is detecting plane in image.



S. Agarwala, L. Jin, C. Rockwell, D.F. Fouhey. *PlaneFormers: From Sparse View Planes to 3D Reconstruction*. ???, 2022

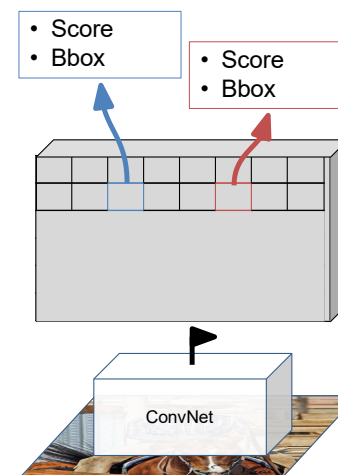
YOLO



- No proposals
- Predict at each location in 7x7 feature map, score for each class + 2 bboxes
- 7x faster than Faster-RCNN, but worse accuracy, precision
- Immensely popular in robotics
- Loads of similar methods (YOLOv2, YOLOv3)

J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, [You Only Look Once: Unified, Real-Time Object Detection](#), CVPR 2016

YOLO

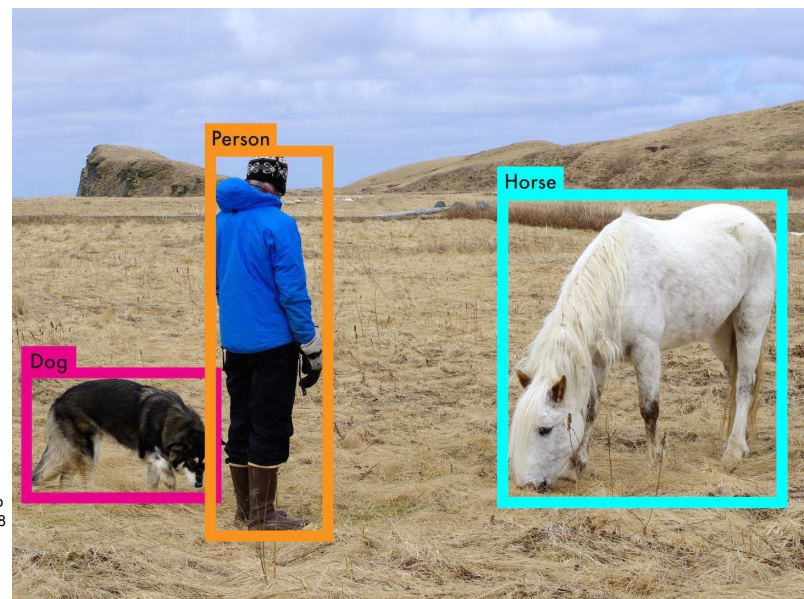


- No proposals
- Predict at each location in 7x7 feature map, score for each class + 2 bboxes
- 7x faster than Faster-RCNN, but worse accuracy, precision
- Immensely popular in robotics
- Loads of similar methods (YOLOv2, YOLOv3)

J. Redmon, A. Farhadi, *YOLOv3: An Incremental Improvement*.



https://docs.google.com/presentation/d/1kAa7NOamBt4calBU9iHgT8a86RRHz9Yz2oh4-GTdX6M/edit#slide=id.g151008b386_0_57



CV 14: Object Detection Algorithms

CV 14: Object Detection Algorithms

Accurate object detection is slow!

	Pascal 2007 mAP	Speed	
DPM v5	33.7	.07 FPS	14 s/img

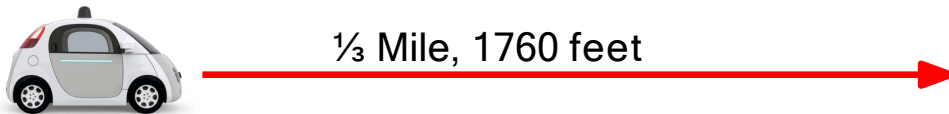
CV 14: Object Detection Algorithms

Accurate object detection is slow!

	Pascal 2007 mAP	Speed	
DPM v5	33.7	.07 FPS	14 s/img
R-CNN	66.0	.05 FPS	20 s/img

Accurate object detection is slow!

	Pascal 2007 mAP	Speed	
DPM v5	33.7	.07 FPS	14 s/img
R-CNN	66.0	.05 FPS	20 s/img



Accurate object detection is slow!

	Pascal 2007 mAP	Speed	
DPM v5	33.7	.07 FPS	14 s/img
R-CNN	66.0	.05 FPS	20 s/img
Fast R-CNN	70.0	.5 FPS	2 s/img
Faster R-CNN	73.2	7 FPS	140 ms/img



Accurate object detection is slow!

	Pascal 2007 mAP	Speed	
DPM v5	33.7	.07 FPS	14 s/img
R-CNN	66.0	.05 FPS	20 s/img
Fast R-CNN	70.0	.5 FPS	2 s/img



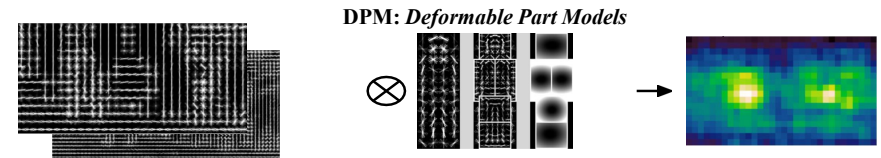
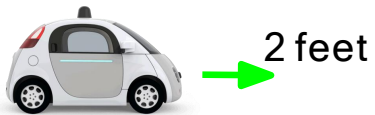
Accurate object detection is slow!

	Pascal 2007 mAP	Speed	
DPM v5	33.7	.07 FPS	14 s/img
R-CNN	66.0	.05 FPS	20 s/img
Fast R-CNN	70.0	.5 FPS	2 s/img
Faster R-CNN	73.2	7 FPS	140 ms/img
YOLO	63.4	45 FPS	22 ms/img



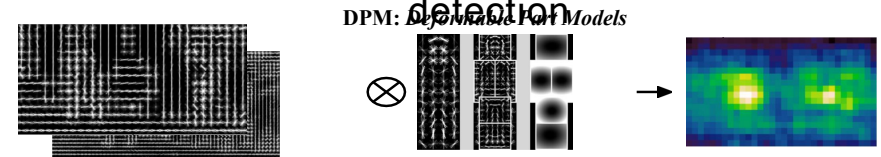
Accurate object detection is slow!

	Pascal 2007 mAP	Speed	
DPM v5	33.7	.07 FPS	14 s/img
R-CNN	66.0	.05 FPS	20 s/img
Fast R-CNN	70.0	.5 FPS	2 s/img
Faster R-CNN	73.2	7 FPS	140 ms/img
YOLO	63.4 69.0	45 FPS	22 ms/img

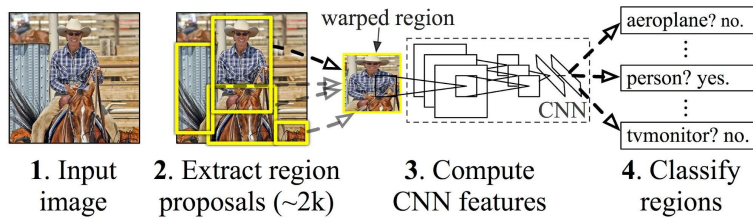


Sliding window, DPM, R-CNN all train region-based classifiers to perform

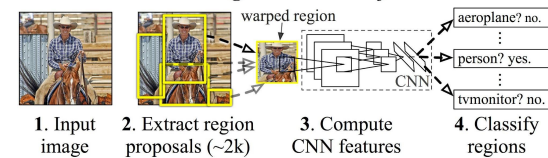
detection

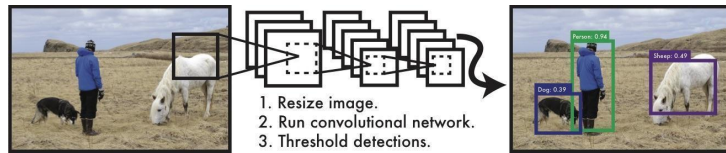


R-CNN: Regions with CNN features

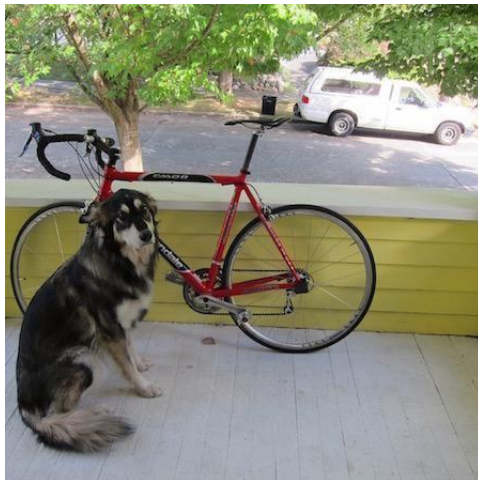
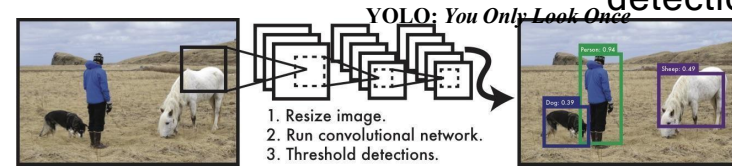


R-CNN: Regions with CNN features

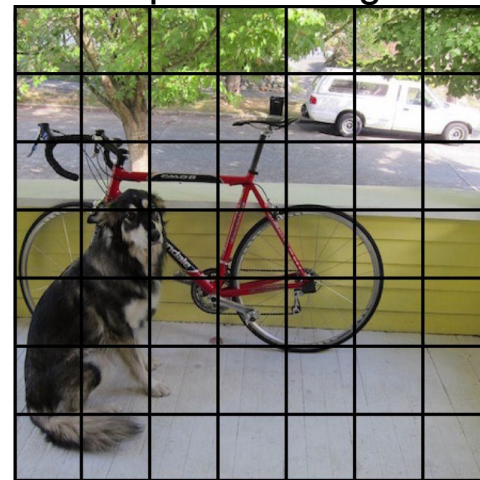




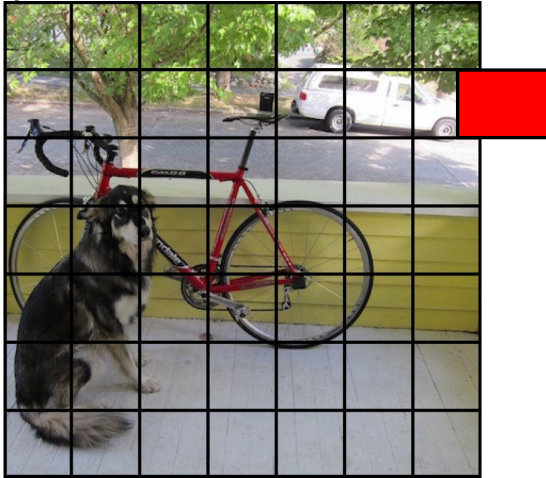
With YOLO, you only look once at an image to perform detection



We split the image into a grid



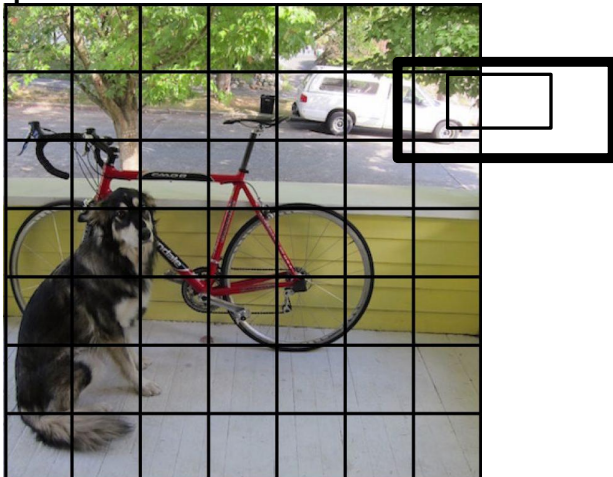
Each cell predicts boxes and confidences: $P(\text{Object})$



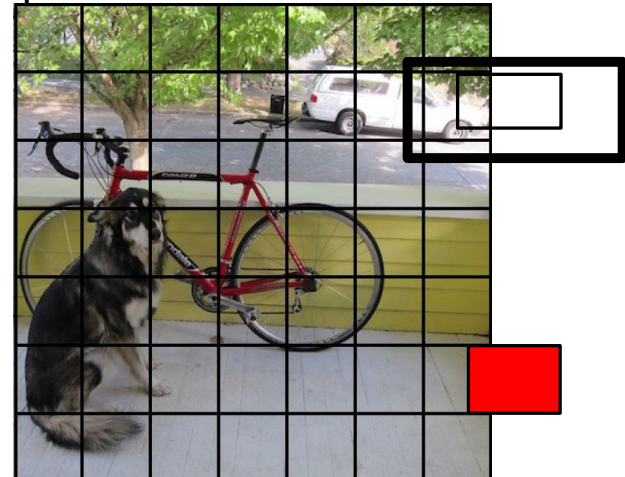
Each cell predicts boxes and confidences: $P(\text{Object})$



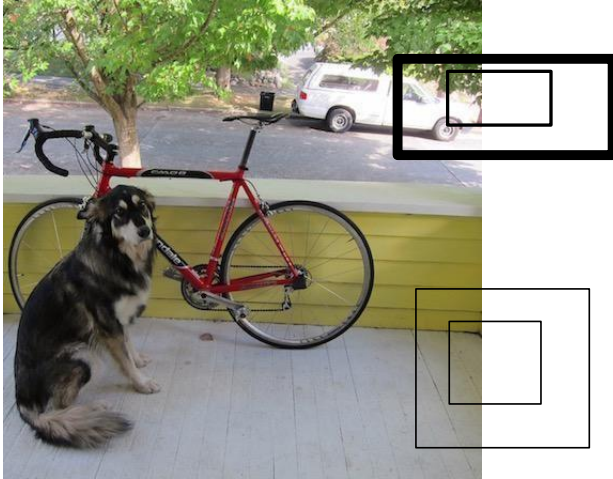
Each cell predicts boxes and confidences: $P(\text{Object})$



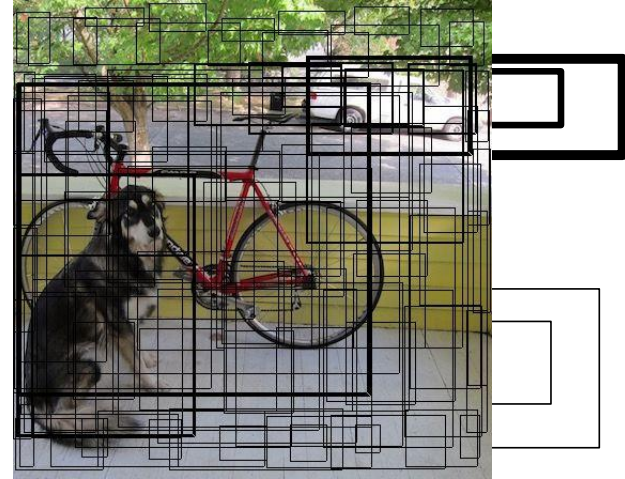
Each cell predicts boxes and confidences: $P(\text{Object})$



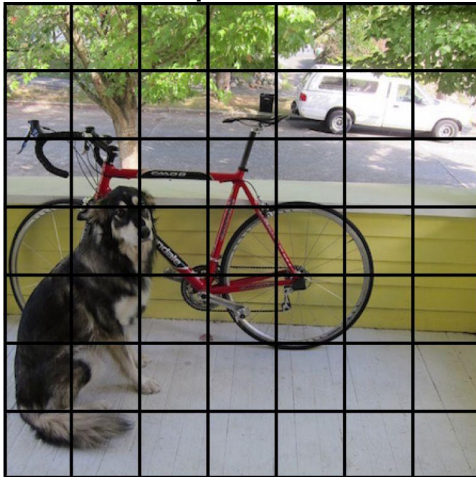
Each cell predicts boxes and confidences: $P(\text{Object})$



Each cell predicts boxes and confidences: $P(\text{Object})$



Each cell also predicts a class probability.



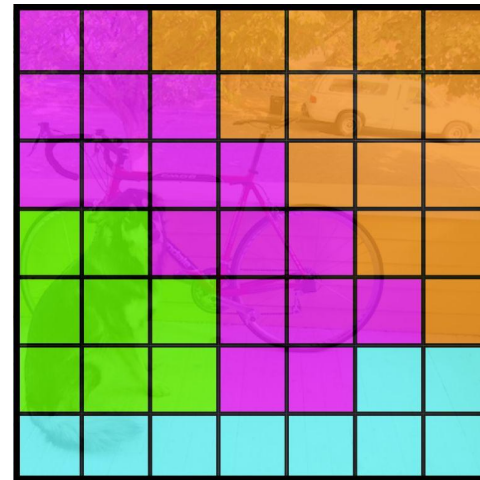
Each cell also predicts a class probability.

Bicycle

Car

Dog

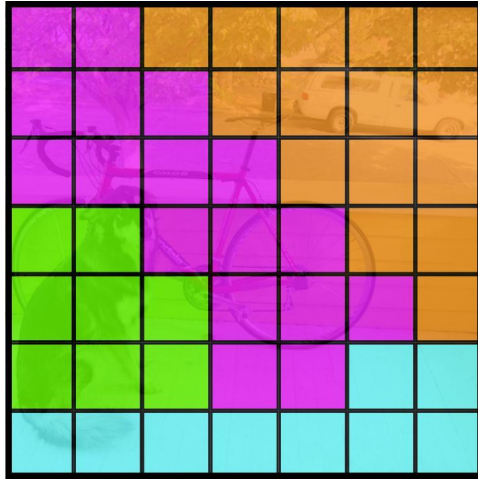
Dining
Table



Conditioned on object: $P(\text{Car} \mid \text{Object})$

Bicycle

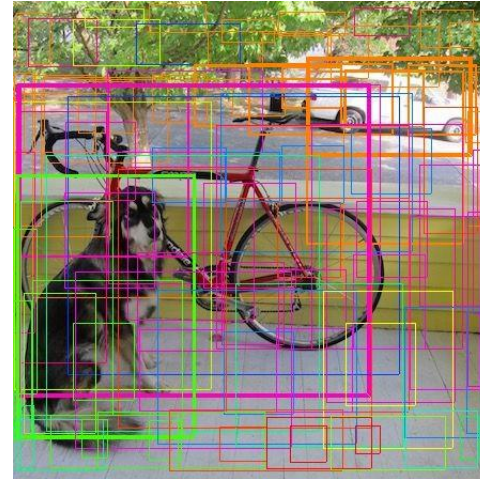
Dog



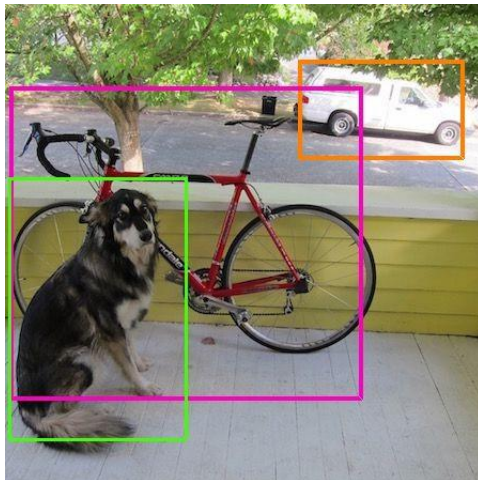
Car

Dining
Table

Then we combine the box and class predictions.



Finally we do NMS and threshold detections



This parameterization fixes the output size

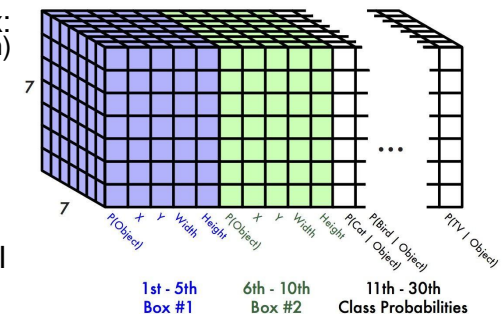
Each cell predicts:

- For each bounding box:
 - 4 coordinates (x, y, w, h)
 - 1 confidence value
- Some number of class probabilities

For Pascal VOC:

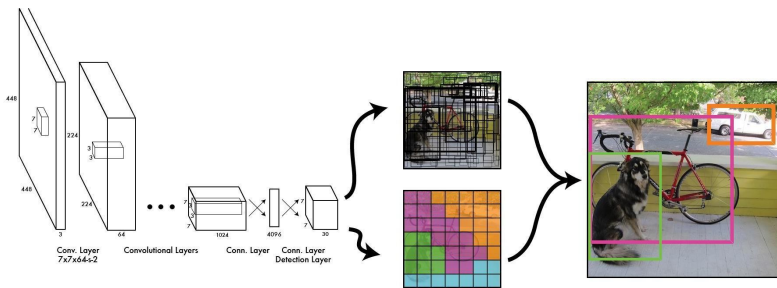
- 7x7 grid
- 2 bounding boxes / cell
- 20 classes

$7 \times 7 \times (2 \times 5 + 20) = 7 \times 7 \times 30$ tensor = **1470 outputs**

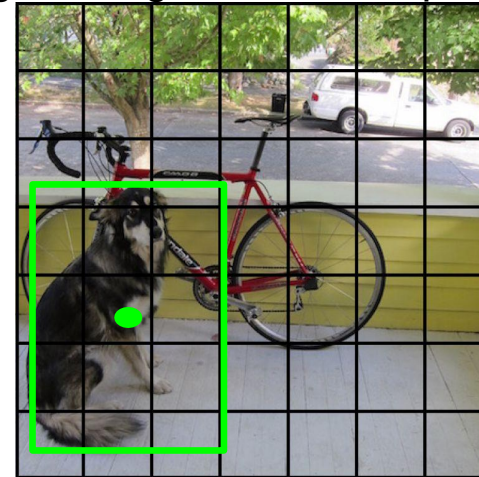


Thus we can train one neural network to be a whole detection pipeline

CV 14: Object Detection Algorithms

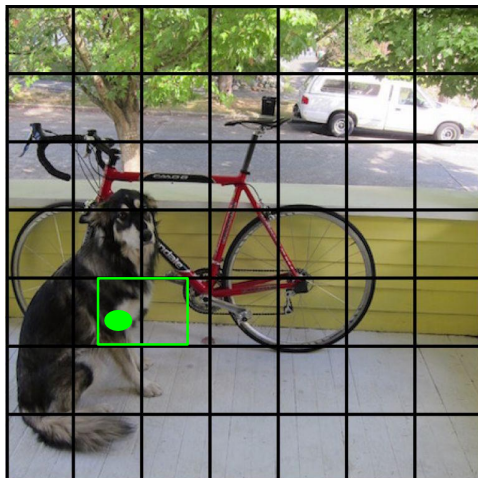


During training, match example to the right cell



CV 14: Object Detection Algorithms

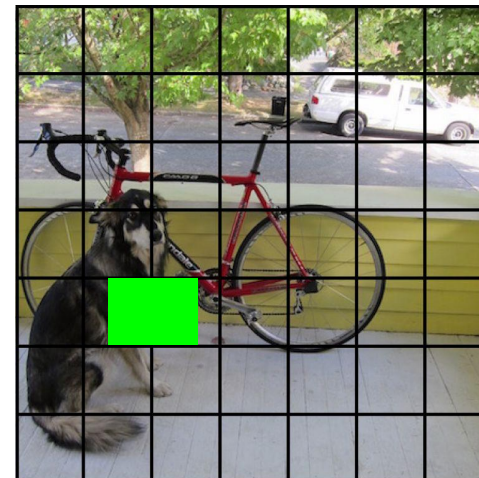
During training, match example to the right cell



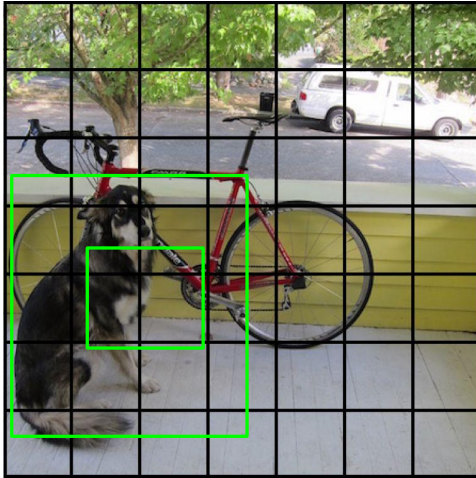
CV 14: Object Detection Algorithms

Adjust that cell's class prediction

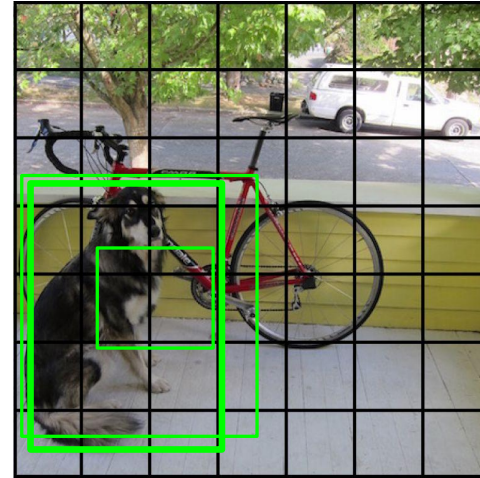
Dog = 1
Cat = 0
Bike = 0
...



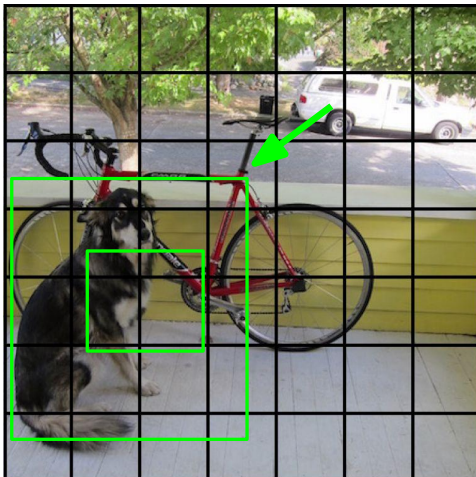
Look at that cell's predicted boxes



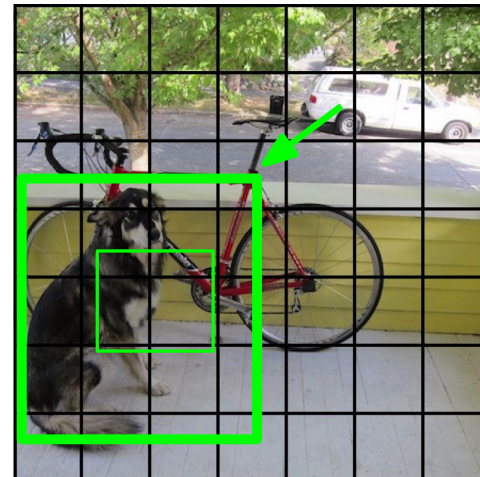
Find the best one, adjust it, increase the confidence



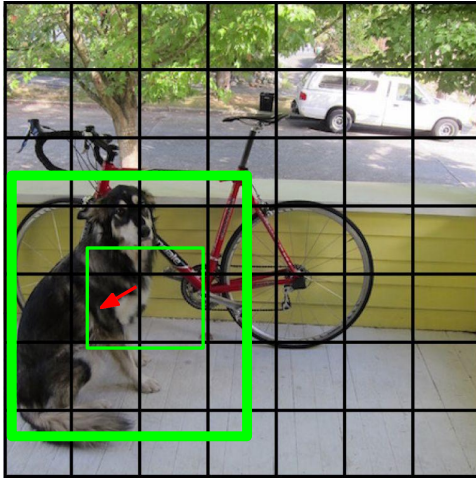
Find the best one, adjust it, increase the confidence



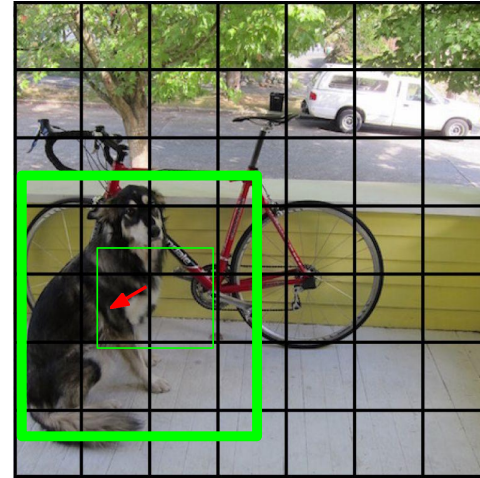
Find the best one, adjust it, increase the confidence



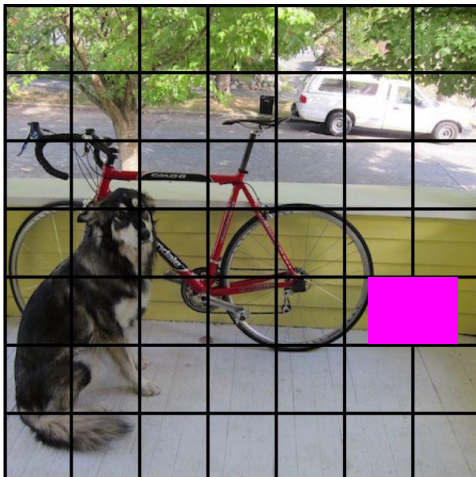
Decrease the confidence of other boxes



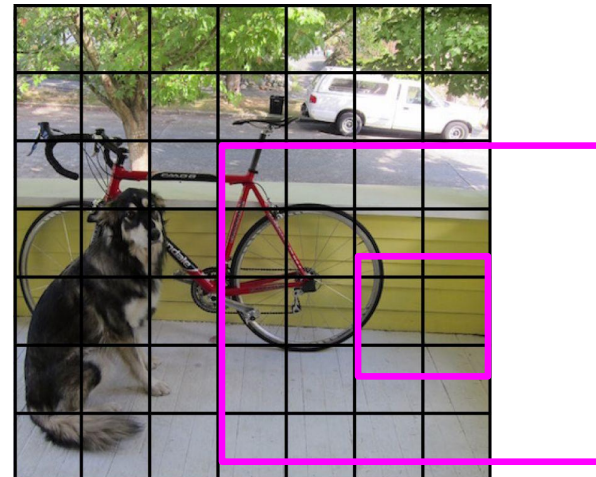
Decrease the confidence of other boxes



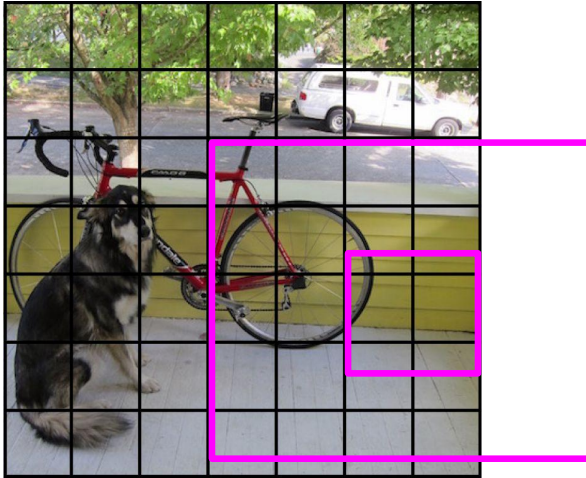
Some cells don't have any ground truth detections!



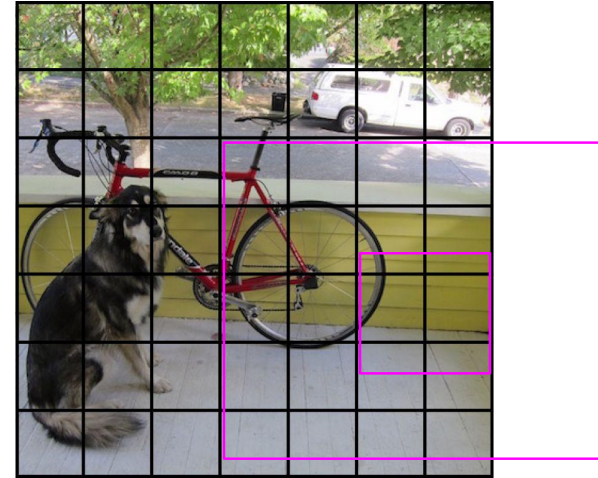
Some cells don't have any ground truth detections!



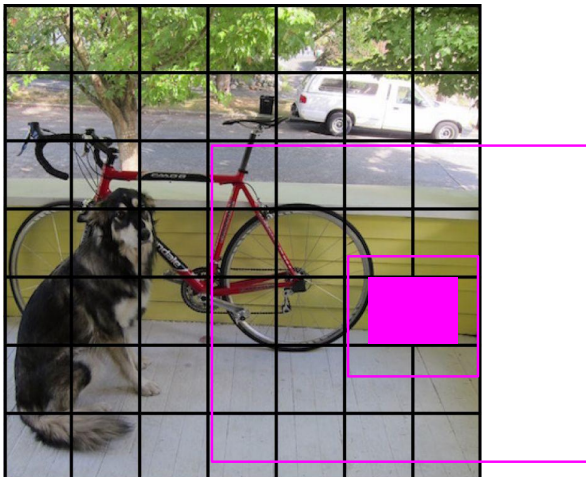
Decrease the confidence of these boxes



Decrease the confidence of these boxes

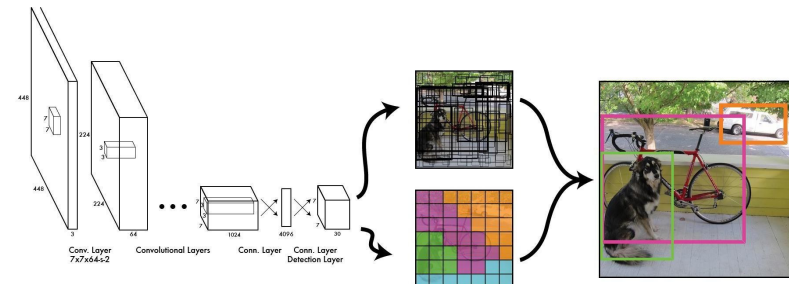


Don't adjust the class probabilities or coordinates



We train with standard tricks:

- Pretraining on Imagenet
- SGD with decreasing learning rate
- Extensive data augmentation
- For details, see the paper



YOLO works across a variety of natural images

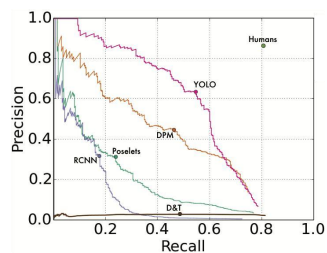


It also generalizes well to new domains (like art)



YOLO outperforms methods like DPM and R-CNN when generalizing to person detection in artwork

Code available! pjreddie.com/yolo

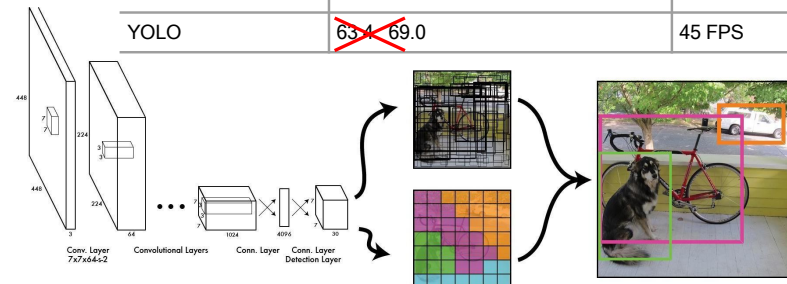


	VOC 2007	Picasso	People-Art
	AP	AP Best F_1	AP
YOLO	59.2	53.3	45
R-CNN	54.2	10.4	26
DPM	43.2	37.8	32

S. Ginosar, D. Haas, T. Brown, and J. Malik. Detecting people in cubist art. In *Computer Vision-ECCV 2014 Workshops*, pages 101–116. Springer, 2014.

H. Cai, Q. Wu, T. Corradi, and P. Hall. The cross-depiction problem: Computer vision algorithms for recognising objects in artwork and in photographs.

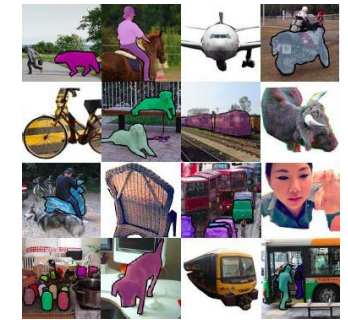
	Pascal 2007 mAP	Speed	
DPM v5	33.7	.07 FPS	14 s/img
R-CNN	66.0	.05 FPS	20 s/img
Fast R-CNN	70.0	.5 FPS	2 s/img
Faster R-CNN	73.2	7 FPS	140 ms/img
YOLO	63.4 69.0	45 FPS	22 ms/img



- 80 categories instead of PASCAL's 20
- Current best mAP: 52%



- ✔ Object segmentation
- ✔ Recognition in context
- ✔ Superpixel stuff segmentation
- ✔ 330K images (>200K labeled)
- ✔ 1.5 million object instances
- ✔ 80 object categories
- ✔ 91 stuff categories
- ✔ 5 captions per image
- ✔ 250,000 people with keypoints



102

CV 14: Object Detection Algorithms

-



CV 14: Object De



**Who takes photos of
open refrigerators
?????**



Guess the category!

These were detected with >90% confidence, corresponding to 99% precision on original dataset

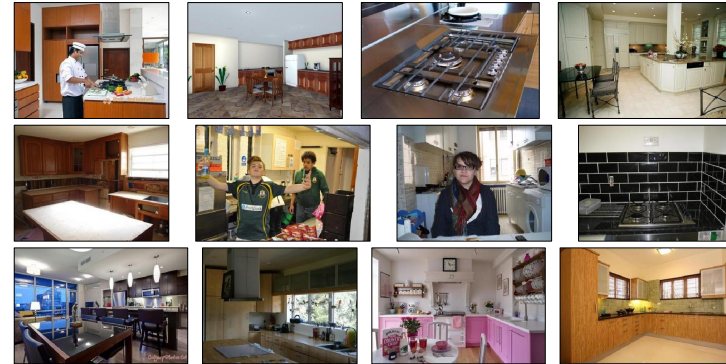


(1) Person

(2) Giraffe

(3) Bicycle

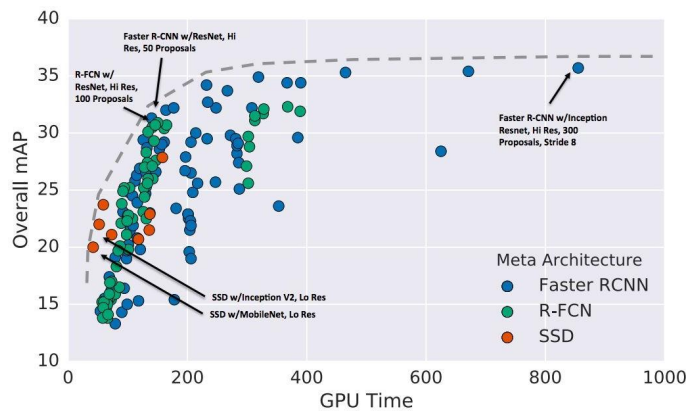
Kitchens from Googling



Places 365 Dataset, Zhou et al. '17

New detection benchmark: COCO (2014)

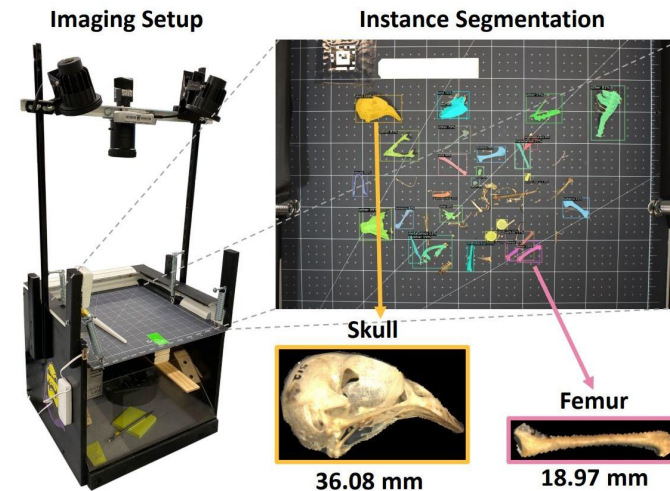
CV 14: Object Detection Algorithms



J. Huang et al., [Speed/accuracy trade-offs for modern convolutional object detectors](#), CVPR 2017

What's It Good For?

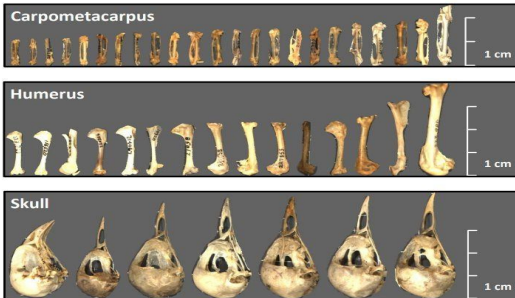
CV 14: Object Detection Algorithms



Z. Zhou, G. Hassena, B.C. Weeks, D.F. Fouhey. *Quantifying Bird Skeletons*. CVPR Workshops, 2021.
 B.C. Weeks, Z. Zhou, B.K. O'Brien, R. Darling, M. Dean, T. Dias, G. Hassena, M. Zhang, D.F. Fouhey. *A deep neural network for high throughput measurement of functional traits on museum skeletal specimens*. Methods in Ecology and Evolution, 2022.

What's It Good For?

CV 14: Object Detection Algorithms



- UM has 25K bird skeletons but measuring bird bones by hand is hard and tedious.
- New solution: dump the bones, take a picture, use a deep network.
- Now enabling testing hypotheses about birds at huge scale

Z. Zhou, G. Hassena, B.C. Weeks, D.F. Fouhey. *Quantifying Bird Skeletons*. CVPR Workshops, 2021.
B.C. Weeks, Z. Zhou, B.K. O'Brien, R. Darling, M. Dean, T. Dias, G. Hassena, M. Zhang, D.F. Fouhey. *A deep neural network for high throughput measurement of functional traits on museum skeletal specimens*. Methods in Ecology and Evolution, 2022.

- (1) Computer Vision: Algorithms and Applications, 2nd ed (Text Book-1), Richard Szeliski ; Section 6.3
- (2) [Object Detection with Deep Learning: A Review](#) [IEEE Transactions on NN & LS)
- (3) [Chapter -10 - Object Detection, Yolo and their Implementation](#)

111



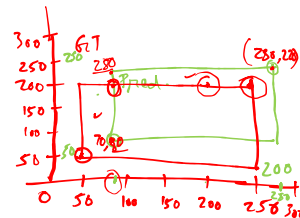
CV 13: Object Detection Algorithms - RCNN

- (1) Computer Vision: Algorithms and Applications, 2nd ed (Text Book-1), Richard Szeliski ; Section 6.3
- (2) [Object Detection with Deep Learning: A Review](#) [IEEE Transactions on NN & LS)
- (3) [Chapter -10 - Object Detection, Yolo and their Implementation](#)
- (4) R-CNN paper - <https://arxiv.org/pdf/1311.2524>
- (5) Fast R-CNN - <https://arxiv.org/pdf/1504.08083>
- (6) Faster R-CNN - <https://arxiv.org/pdf/1506.01497>

67

Numericals

GT BBox $\rightarrow (x_1, y_1) \quad (x_2, y_2)$
 Bottom Left Top Right
 $(50, 50, 250, 200)$



Pred BBox $\rightarrow (70, 80, 280, 220)$.

Calculate the IOU? $IOU = \frac{\text{Intersection Area}}{\text{Union of Area}}$

$$\begin{aligned} \text{Union of Area} &= \text{Area of GT} + \text{Area of Pred} - \text{Area of Overlap} \\ &= \overset{ht}{(200-50)} \times \overset{width}{(250-50)} + \overset{ht}{(220-80)} \times (280-70) - 21600 \\ &= 30000 + 29400 = 21600 \end{aligned}$$

$$\text{Intersection Area} = \max(30, 70), \max(50, 80) = 170, 80$$

$$\min(250, 280) \min(200, 220) = (250, 200)$$

$$\text{Intersection height} = (200 - 80) = 120$$

$$\text{Intersection width} = (250 - 70) = 180$$

$$\text{Intersection Area} = 120 \times 180 = 21600$$

$$\text{Union Area} = 37800$$

$$\text{IOU} = \frac{21600}{37800} = 0.57$$

$$\text{Dice Coeff} = \frac{2 \times \text{IOU}}{\text{Total Area}} = \frac{2 \times 21600}{30000 + 25400} = 0.7272$$

single object detection $\frac{TP, FP, FN, TN}{mAP} \rightarrow 9 \text{ point} \rightarrow 11 \text{ point}$

car. GT $\sqrt{5}$ cars. has

model predicts $\boxed{10}$ $\rightarrow$ CF IOU's - $P = TP / (TP + FP)$

Rank	TP/FP	Comm Precision	Recall
1	TP	1/1 = 1	1/5 = 0.2
2	TP	2/2 = 1	2/5 = 0.4
3	FP	2/3 = 0.67	2/5 = 0.4
4	TP	3/4 = 0.75	3/5 = 0.6
5	FP	3/5 = 0.6	3/5 = 0.6
6	TP	4/6 = 0.67	4/5 = 0.8
7	TP	5/7 = 0.71	5/5 = 1
8	FP	5/8 = 0.63	5/5 = 1
9	FP	5/9 = 0.56	5/5 = 1
10	TP	5/10 = 0.5	5/5 = 1

Caveat * Even though greater than IOU but we have already detected it based on CS

Interpolated Precision at 9 recall levels.
 $P_{inter} = \max P(r')$ for $r' \leq r$ for all r .

0.1	Recall > 0.1	Precision
0.2	Recall = 0.2	1
0.3	0.3	1
0.4	0.4	0.75
0.5	0.5	0.75
0.6		0.71
0.7		0.71
0.8		0.71
0.9		0.71
1		0.71

$$mAP = \frac{0.8 + 0.9 + 0.1}{1000} =$$

$$\begin{aligned} \text{9pt mAP}_{\text{car.}} &= \frac{1}{9} \sum P_{inter}(r) \\ &= \frac{1}{9} \times (1 + 1 + 1 + 1 + 0.75 + 0.75 + 0.71 + 0.71 + 0.71) \\ &= \frac{7.63}{9} \approx 0.8 \end{aligned}$$